

fmix layer 1: the round menu

The per-round move menu as implemented

28 July 2026

Abstract

This describes `fmix`'s per-round move menu as it now stands, after the restructure of 27–28 July. It is a reference for what the system does, not a design history; where a measurement has since argued against a design choice, the measurement is stated inline. The phase overlay — the rule engine that changes parameters mid-run — is layer 2 and does not exist yet. Design rationale is in `docs/FMIX_MENU.md`; the ancestry measurements are in `docs/ANCESTRY_INSTRUMENTATION.pdf`.

1 The round

Every move is one round. Four slots, evaluated in order; the first that fires consumes the round.

#	slot	fires with	effect on size
0	conditions	always	none
1	twist	p_{twist}	+2 / +6 before absorption
2	DB move	p_{db}	depends on <code>db_mode</code>
3	thermostat	otherwise	$\pm$ small

Slot 3 runs *iff the slot-2 coin did not fire*. A slot-2 round whose descent finds nothing is spent — there is no fallthrough — so the thermostat receives exactly $(1 - p_{\text{twist}})(1 - p_{\text{db}})$ of rounds regardless of how hard the material is.

Slot 0. Two hardcoded conditions: the pool rebuild on its cadence, and the size brake (§7). The stop checks — flag, canary, dose — run at the report point. A general condition→action rule engine is layer 2.

2 The DB move

One operation with three admission rules. The rule is selected by `db_mode` at slot 2, and fixed by role on the thermostat branches.

2.1 Window selection and the descent

Draw a seed gate (§5); sample s_{db} gates around it, convex with probability p_{convex} else contiguous. Both gather until the block is contiguous in the arena. A gate with $\geq w_{\text{window}}$ controls is ineligible; a window containing one is truncated before it.

The descent then tries the window and, on failure, drops one gate and retries, down to length 1. **The gate dropped is the one farthest from the seed**, so the seed survives to the shortest rung

— dropping from a fixed end instead walks away from the very gate the descent exists to re-encode, on roughly half of contiguous windows and *every* convex one.

The bottom rung always pays. A single gate’s permutation has exactly one one-gate implementation: itself. So at length 1 the free branch is unreachable and MIX-DB either finds nothing or pays the smallest non-identical spelling. Every descent therefore ends in a paid expansion or a genuine miss, which is what makes per-gate miss accounting unnecessary. Measured: the length-1 rung is the busiest of all, carrying 24% of splices.

2.2 The identity guard

A candidate identical to the outgoing window, gate for gate, is never spliced, in all three modes. Splicing one is a no-op that still costs a round and still stamps a generation — a re-encoding the dose meter would count and that did not happen. Measured: it refuses roughly **one candidate in six**.

2.3 The three modes

mode	admission	selection
ANY-DB	any size	uniform over all non-identical equivalents
MIX-DB	free if possible, else pay	uniform over non-identical $\leq$ window; else uniform over the smallest
COMP-DB	only if some match $\leq$ window	uniform over the minimum-size ones

The asymmetry between MIX and COMP is deliberate: COMP is a contraction move so minimum size is its job, while MIX is a re-encoding move so entropy is. MIX-DB makes the ingest-versus-pay decision *per window*, from the match list the lookup already returned — which is what replaced an entire per-gate tier state machine.

Where each is used. Slot 2 uses `db_mode`, a deterministic three-way flag. The contract branch is always COMP-DB (it must not grow); the expand branch is always ANY-DB (it exists to grow).

Selection is lazy. Every rule is a function of gate counts, and the stored value is a flat length-prefixed sequence, so candidates are catalogued by walking offsets and *only the winner is decoded*. A candidate that fails to place, or turns out to be the window, is dropped and the choice retried.

2.4 The curated store

Optional (`-curated`), off by default. Curated entries are halves of split minimal identities, so their internal pieces are not locally compressible — a route `fcompress` cannot partially undo. Curatedness is a lexicographic first key: a non-identical curated match beats a regular one regardless of size, with the mode’s own rule applied inside the winning class. COMP is exempt.

Forward key only. Probing curated on the reverse canonical key returns entries belonging to a *different* permutation. A store-level probe of one window found 430,568 curated candidates, all from the reverse key, *none* equivalent; the forward key returned none, and the regular store returned 6, all equivalent. A curated replacement that still fails verification is refused and counted rather than fatal, and `-curated` refuses `-no-db-verify`.

Status: not runnable. The store’s fan-in per key is enormous, and even with lazy decoding the catalogue walk is linear in it. Curated runs are roughly 30× slower than uncurated and have not completed.

3 Twists

Slot 1 is the only twist channel. Type is drawn from the three `w_twist_*` weights as ratios; window length is log-uniform over `[twist_min_len, |C|]` with the virtual start drawn symmetrically, so head and tail accumulate coverage equally.

Placement. A bracket dropped at random must be paid for; one dropped next to a gate that can swallow it costs nothing. `TWIST_PATTERNS` is a table of welcoming shapes; the placer samples up to `twist_place_tries` candidate positions and falls back to the random draw otherwise, reporting `tplace=placed/fallback` so an inert table is visible. One entry so far: a `comp=1` gate absorbs a NOT on its target and comes out `comp=0`, eroding a fossil in the bargain. That absorption needed a `merge_result` extension — it is always mathematically legal, but the catalogue previously refused it at width ≥ 2 .

Recorded, unconsumed. A verified hidden-swap identity, in g57 notation $[x, y, z] = x \hat{=} y \vee \neg z$:

$$[a, b, c] \cdot \text{swap}(a, b) \cdot [b, c, a] = [b, a, c] \cdot [b, c, a] \cdot [a, b, c] \cdot [a, c, b]$$

checked exhaustively on all 8 inputs. Two g57s bracketing a swap equal four g57s and *no swap*, so a swap conjugation sited between a matching pair costs +2 gates rather than +6 and leaves no swap-shaped fingerprint. Consuming it needs a rewrite path, which is a different operation from siting a bracket.

Cost. Measured at 500k moves: the interior relabel costs **0.4%** of runtime at $p_{\text{twist}} = 0.002$, and 28 ns per gate-visit. A deferred-relabel ledger was designed and then dropped on that evidence.

4 The thermostat

$$p(\text{contract}) = \sigma\left(\frac{\text{size} - \text{target_size}}{\text{temp}}\right), \quad \text{clamped to } [0.02, \text{contract_ceiling}]$$

`target_size` sets where the equilibrium sits; `temp` is the width of the transition band, not a rate — small `temp` gives tight regulation and fast approach, large gives breathing.

Contract tries COMP-DB with probability p_{comp} , then journal undo, then the merge catalogue. **Expand** is an ANY-DB move with probability p_{any} , else a cross. Unsubsume, insert and fresh-split are retired.

Two consequences worth stating. The contraction ceiling stays at 0.98: its 2% expansion floor is a structural growth source, but it is also what keeps crossings — hence fossil erosion — running at target, and tightening it to 0.9995 starves erosion by 40×. And with the catalogue-invertible expansions gone, **deep contraction is COMP-DB’s job**: crossing ladders are not pairwise-recoverable, so without a store the drain has only the journal.

Caveat. At phase-A rates (p_{db} near 1) slot 3 barely runs, so `target_size` and `temp` set the contract/expand mix rather than the size. Size control there comes from the brake.

5 Seeding and the generation pool

Every gate carries a generation: inputs start at 0, a splice stamps the outgoing window’s upper-median +1, splits stamp parent +1, merges take the min, twist brackets are born-random.

Seed selection is one coin: with probability p_{mingen} draw from the *generation pool*, else uniformly. The pool is the `pool_k` lowest-generation gates that are **pool-eligible** ($< w_{\text{pool}}$ controls) **and below the goal**. Both filters carry weight: an ineligible gate can never be re-encoded, so an unfiltered pool converges on exactly that set; and without the below-goal filter a late pool fills with ordinary finished gates and the canary can never fire.

`pool_k` is a *count*, not a fraction, because the drain rate is set by the move economy and is independent of circuit size.

Two eligibility thresholds. w_{window} governs what may sit *inside* a window; w_{pool} what may *seed* one and count toward the dose. They differ because width-3 gates match in context often enough to admit to windows but re-encode end-to-end at 0.41% against 98.98% for width ≤ 2 , so at a shared threshold they accumulate in the pool with nothing to eject them.

6 The canary

Over a ring buffer of the last `canary_window` *qualifying* rounds, the fraction that failed at every rung. A round qualifies only if its seed genuinely came from the pool; a heads coin that fell through a drained pool is counted separately, because that means the rebuild is too slow rather than that the material is unreachable — opposite remedies. The condition fires when the buffer is full and the fraction exceeds θ , and it stops the run (`MixStop::CanaryFired`), checked ahead of the dose stop.

It **sleeps while the brake holds COMP**: COMP declines far more often by construction, and since this is a stop condition those samples would end runs for a reason unrelated to reachability.

7 Size control and breathing

Growth past `size_hi` forces slot 2 into COMP; it is released at `size_lo` or when COMP’s shed rate over a trailing window falls below `comp_release_eps`, whichever comes first.

The productivity release is what makes a *wide* band safe, and a wide band is what transport wants, since growth legs are where material moves. The danger was never width but sitting in COMP past its usefulness, where it starves as the circuit nears local minimality and spends re-encoding diversity — COMP draws only from minimum-size spellings, pulling toward exactly the form `fcompress` would compute anyway.

Measured, manually. A 2M-move exhale at $s_{db} = 12$ shed -0.060 gates/move against -0.054 at $s_{db} = 9$ — **COMP wants a longer window than MIX**. Mixing did *not* degrade: `anc`, `span` and litter diversity all rose. The only cost was `dmin`, $0.168 \rightarrow 0.136$. No bend in the shed rate appeared in 2M moves.

8 Litter rules

Every gate carries a litter id — the replacement event that created it — and its litter’s birth size. Splits propagate the id, a splice mints a fresh one, a merge unions. Two optional rules, both off by default and neither yet A/B’d: `-litter-ban` refuses a rung that is exactly one complete litter (the

unit an earlier replacement emitted, and so where the store can hand that spelling straight back), and `-litter-samples` draws several candidate windows and keeps the most litter-diverse.

Measured: at equilibrium roughly 70 full-litter splices per 100k moves, and windows drawing from only ~ 3 distinct litters against a ceiling of s_{db} — reached almost immediately, not a transient. That gap is the largest visible untouched lever.

9 Instrumentation

Beyond the long-standing report line:

field	meaning
<code>idsk</code>	candidates refused by the identity guard
<code>cur</code>	curated hits / curated rejections
<code>g57only</code>	g57-only COMP attempts and hits (prints hits / rounds)
<code>dmin</code>	fraction of windows admitting a strictly shorter spelling
<code>osyn</code>	fraction of gates whose ancestry label is destroyed
<code>anc, ancspan</code>	mean ancestor-set size and span
<code>distinct, full, ban</code>	litter diversity, complete-litter splices, bans
<code>tplace</code>	twists placed on a pattern / fallen back
<code>canary, cft</code>	canary failure fraction, pool fall-throughs
<code>g57</code>	true g57 census (comp=1, two opposite-polarity controls)
<code>shaped</code>	the same census with polarity ignored — see below
<code>polf</code>	$(\text{shaped} - \text{g57})/\text{shaped}$: the twist odometer

Reading shaped against g57. These two answer different questions and the single `g57` field used to conflate them. The store emits `g57` circuits and nothing else — measured directly, with twists disabled `g57/comp` holds at 1.000 for a whole run — so `shaped` is the DB-effectiveness reading: $1 - \text{shaped}/\text{size}$ is exactly the material the DB did not produce.

`polf` is a twist dose, not erosion. A negation twist conjugates its window by NOT on one wire, flipping the polarity of every literal on it. A `g57`'s two controls lie on distinct wires, so a twist touches at most one, and touching one flips the pair from opposite to same: the gate keeps its `comp`, its width and its place, and only stops matching the strict shape. Swap twists carry polarity with the wire and leave `polf` at zero. A/B at equal twist count over 400k moves gives `g57/comp` = 1.000 (no twists), 0.704 (neg), 1.000 (swap, at $2.6\times$ the relabel count), while `comp/size` never leaves 0.996.

Unlike `cov`, whose denominator is the growing circuit, `polf` is bounded in $[0, \frac{1}{2}]$ and cannot fall while twists keep firing.

Plus two extra lines: the **splice size histogram** (joint outgoing $\rightarrow$ incoming length, which the scalar totals cannot show) and, under `-ancestors`, the **ancestry report** with log-bucketed histograms of `anc`, `span` and fan-out.

Do not quote odiff, oadj or disp without osyn. All three skip gates whose ancestry label a mixed-lineage splice destroyed, so they are computed over the material mixing has failed to touch and grow more selective the better it works. `osyn` reaches 50.7% by 1M moves and 1.000 on small circuits, where those three are undefined rather than merely degraded.

10 Checkpoint and resume

`-state-out` writes a resume file at every stop, and one is written beside each `-snap-every-moves` snapshot. `-resume` rebuilds the chain; `-input` is then not required.

The circuit file alone cannot do this. Per-gate direction has no sidecar and the whole directional walk rides on it; the undo journal references arena ids, so the checkpoint rennumbers to arena order and remaps, dropping entries with dead pieces; and the *original* circuit is what `global_check` verifies against, so a resumed run without it would verify nothing about fidelity to the true input. Also carried: moves, event and litter counters, the tabu queue, `twspan`, the canary ring, `db_mode_cur` and brake state, the pool, ancestor sets, and every counter. `StdRng` is not serialisable, so a fresh seed is drawn and stored — a clean continuation, not a bit-identical replay. The file is version-stamped and refuses to load across versions.

A resume takes every parameter from the command line, including `-db-mode` and `-p-mingen`. The saved mode is diagnostic only.

11 What is not built

- The slot-0 rule engine. The brake and canary are hardcoded.
- `twist_db_mode` and DB absorption of CNOT packets; the hidden-swap identity is verified but unconsumed, and wants table entries to be rewrite rules rather than site hints.
- A bounded candidate walk for curated, without which it cannot run.
- Store guards read from store metadata rather than retyped per run.
- A forced pool rebuild on `COMP` $\rightarrow$ `MIX`.